B23 PyTorch foundations

Tensors, autograd, gradient descent, nn.Module, loss functions and data loading, built up from NumPy.

B23-L1 Tensors, shape, dtype and shared memory

Convert to float32 at the edge, know which operations share memory, and make predictions and targets the same shape before you compare them.

```
x = torch.zeros(4, 3)
x.shape                          # torch.Size([4, 3])
x.sum(dim=0).shape               # torch.Size([3])
x.sum(dim=1, keepdim=True).shape # torch.Size([4, 1])
x.unsqueeze(0).shape             # torch.Size([1, 4, 3])
x.reshape(-1).shape              # torch.Size([12])
```

PyTorch and NumPy default floats	PyTorch float32, NumPy float64. Convert with torch.as_tensor(X, dtype=torch.float32) at the edge.
What shares memory	torch.from_numpy, view and slices share memory. torch.tensor copies. In-place ops on a shared tensor change the source.
(n, 1) against (n,)	Broadcasts to (n, n) without an error. Flatten with squeeze(1) or reshape(-1) and assert shapes.
Shape after sum(dim=1, keepdim=True) on (4, 3)	(4, 1). Without keepdim: (4,).

B23-L2 Autograd, gradients from the chain rule

backward() fills .grad on leaf tensors. Update in place under no_grad, reset .grad, and check any hand-written gradient with central differences.

```
w = torch.tensor(3.0, requires_grad=True)
y = w ** 2 + 2 * w
y.backward()
w.grad.item()          # 8.0, since dy/dw = 2w + 2
```

What backward() does	Applies the chain rule from a scalar loss and stores gradients in .grad of leaf tensors (created with requires_grad=True).
Safe manual update	with torch.no_grad(): w -= lr * w.grad, then w.grad.zero_().
Gradient of mean squared error for a linear model	2 / n * X.T @ (X @ w - y).
Central-difference check	(f(w + h) - f(w - h)) / (2 * h) per weight. Disagreement beyond about 1e-6 means the formula is wrong.

B23-L3 Gradient descent, learning rates and optimisers

Step against the gradient, scale features so one rate fits all weights, watch the loss curve, and let an optimiser such as AdamW hold the update rule.

```
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)

optimizer.zero_grad()    # reset .grad on every parameter
loss.backward()          # fill .grad (lesson B23-L2)
optimizer.step()         # apply the update
```

The gradient descent update	w = w - lr * grad. Against the gradient, scaled by the learning rate.

Signs of a bad learning rate	Too high: loss grows or turns into nan. Too low: loss barely falls.
One optimiser step in PyTorch	optimizer.zero_grad(), loss.backward(), optimizer.step().
AdamW and SGD defaults in PyTorch 2.15	AdamW weight_decay 0.01, SGD weight_decay 0. A good start: AdamW with lr 1e-3.

B23-L4 Models as nn.Module, layers, activations and parameters

Subclass nn.Module, call `super().__init__()` first, register layers in attributes, Sequential or ModuleList, and put activations between Linear layers.

```python
class Recommender(nn.Module):
    def __init__(self, n_features: int, hidden: int) -> None:
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_features, hidden),
            nn.ReLU(),
            nn.Linear(hidden, 1),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.net(x)
```

Skeleton of a PyTorch model	Subclass nn.Module, call `super().__init__()` first, build layers in `__init__`, use them in forward, call model(x).
nn.Linear(in, out) weight shape and parameter count	Weight (out, in), bias (out,). Parameters: in * out + out.
Why put ReLU between Linear layers	Two linear layers equal one. A non-linear activation lets depth fit curves and interactions.
Layers in a plain Python list	Not registered: parameters() misses them and they never train. Use nn.ModuleList or nn.Sequential.

B23-L5 Loss functions, logits and target shapes

Regression: MSELoss on matching shapes. Binary: logits into BCEWithLogitsLoss with float targets and no sigmoid in the model. Squeeze (n, 1) outputs first.

Loss for a numeric target	nn.MSELoss() on predictions and targets of the same shape. Squeeze (n, 1) outputs first.
Loss for a 0 or 1 target	nn.BCEWithLogitsLoss() on raw logits and float targets. No nn.Sigmoid() in the model.
pos_weight for an imbalanced binary target	torch.tensor([n_neg / n_pos]), computed on training rows.
Stable binary loss formula	mean(max(z, 0) - z * y + log1p(exp(-|z|))).

B23-L6 Dataset, DataLoader and preprocessing fitted on training rows

A Dataset returns rows, a DataLoader batches them, training shuffles and validation does not, and every preprocessing statistic comes from training rows only.

```python
class SurveyRows(Dataset):
    def __init__(self, X: np.ndarray, y: np.ndarray) -> None:
        self.X = torch.as_tensor(X, dtype=torch.float32)
        self.y = torch.as_tensor(y, dtype=torch.float32)
```

```
    def __len__(self) -> int:
        return len(self.y)

    def __getitem__(self, i: int) -> tuple[torch.Tensor, torch.Tensor]:
        return self.X[i], self.y[i]


train_loader = DataLoader(train_rows, batch_size=64, shuffle=True)
val_loader = DataLoader(val_rows, batch_size=256)
```

The two methods of a Dataset	`__len__` returns the row count. `__getitem__(i)` returns row i as tensors, converted once in `__init__`.
DataLoader batches for 10 rows, batch_size=4	4, 4 and 2. drop_last=True gives 4 and 4.
Which loaders shuffle	Training: shuffle=True with a seeded generator. Validation and test: no shuffle.
Preprocessing rule	Fit means, standard deviations and categories on training rows only. Guard std 0, impute NaN first.